Republic of the Philippines
POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
SAN PEDRO CAMPUS
PUP, Main Road, Barangay United Bayanihan, City of San Pedro, Laguna

High School Event Management System Database Roles and Privileges

Activity 4 Progress Documentation

1. Introduction

This document presents the progress of Activity 4: Database Users, Roles, Privileges, and Access Control for the Group 2 High School Event Management System.

The activity focuses on inspecting database roles, checking existing privileges, and testing access restrictions using PostgreSQL. It also evaluates whether the current permissions follow the principle of least privilege.

2. Project Overview

The High School Event Management System uses PostgreSQL 18 as its database management system.

Component	Details
Database	grp02_event_management_db
Schema	school_event_management_db
DBMS	PostgreSQL 18
Backend	Java 25 – Spring Boot 4.1
Frontend	React 19 – Vite

Database Owner	postgres
Schema Owner	event_owner
Runtime Account	event_app

The database contains 10 tables and 9 sequences. It uses Flyway migrations, with event_owner managing the application schema and its database objects.

3. Database Inspection

The database structure was inspected before checking its access-control configuration.
The inspection confirmed the following:

Database Component	Number
Application Schema	1
Tables	10
Sequences	9

The application schema contains the following tables:

Table Name	Description
attendance	Stores event attendance records.
audit_log	Stores system activity logs.
event	Stores event information.
event_assignment	Stores event assignments.
event_schedule	Stores event schedules.
flyway_schema_history	Tracks database migrations.
participant	Stores participant information.
role	Stores application roles.

user_account	Stores user account information.
venue	Stores venue information.

All 10 tables and 9 sequences are owned by event_owner, while the database itself is owned by postgres.

4. Role Creation and Configuration

The existing PostgreSQL roles were inspected to identify their login capabilities and responsibilities.

Role Name	Login Capability	Description
event_app	LOGIN	Application runtime account.
Event_owner	LOGIN	Owns the application schema and database objects.
app_runtime_grp	NOLOGIN	Existing runtime group role.
app_admin_grp	NOLOGIN	Existing administrative group role.
postgres	LOGIN	PostgreSQL superuser.
activity4_test_role	NOLOGIN	Existing test role

The event_app and event_owner accounts are not members of app_runtime_grp or app_admin_grp. The application also has Super Admin and Event Organizer roles, which are managed by the application and are separate from PostgreSQL roles

5. Privilege Assignment

Privileges were inspected to determine the permissions assigned to each database account. The existing configuration uses direct grants for the event_app account, while event_owner manages the application schema and its objects.

5.1 Database and Schema Privileges

The event_app and event_owner accounts have CONNECT and TEMP privileges on grp02_event_management_db.

The event_app account has USAGE permission on the school_event_management_db schema but does not have CREATE permission. Meanwhile, event_owner has both USAGE and CREATE permissions because it owns the application schema.

5.2 Table Privileges

The event_app account was granted different permissions depending on the purpose of each table.

Its permissions include:

- SELECT on all nine application tables covered by the privilege summary.
- INSERT, UPDATE, and DELETE on tables where these operations are permitted.
- SELECT-only access to the role table.
- SELECT, INSERT, and UPDATE access to the user_account table.
- SELECT and INSERT access to the audit_log table.

The event_owner account owns all application tables and has the corresponding ownership privileges.

5.3 Sequence Privileges

The event_app account has USAGE permission on all nine sequences in the application schema.

It does not have SELECT or UPDATE privileges on these sequences. All nine sequences are owned by event_owner.

5.4 Group Roles

The existing app_runtime_grp and app_admin_grp roles are NOLOGIN group roles.

However, neither role has Group 2-specific schema or table privileges, and neither event_app nor event_owner is a member of these group roles. The current application permissions are assigned directly to the login accounts.

5.5 Default Privileges

- No Group 2-specific default privileges were found in the application schema.

- This means no additional default table or sequence privileges were configured for future objects through pg_default_acl. Future objects may require explicit privilege assignments.

6. Access Control Testing

The runtime account was tested to verify that its assigned privileges worked as expected.

The event_app account was used for normal application operations, while the permissions of event_owner and the existing group roles were inspected to identify their access levels. Access restrictions were also tested to check whether the runtime account could perform unauthorized database operations.

6.1 Runtime Account

The following tests were performed using the event_app account:

Operation	Expected Result	Actual Result
SELECT from event	Allowed	Successful
INSERT into event	Allowed	Successful
UPDATE event	Allowed	Successful
DELETE from event	Allowed	Successful
CREATE TABLE	Denied	Permission denied
TRUNCATE event	Denied	Permission denied
CREATE ROLE	Denied	Permission denied

The SELECT operation successfully retrieved records from the event table. The INSERT operation was tested inside a transaction and rolled back afterward to avoid permanently adding test records.

The UPDATE and DELETE permission tests were also successful, with zero rows affected. This confirmed that the account has the required privileges without changing existing records.

The runtime account was unable to create tables, truncate the event table, or create PostgreSQL roles. These results confirmed that the account can perform permitted application operations while remaining restricted from the tested administrative operations.

6.2 Read-Only Account

- No separate read-only login account or group role was identified in the Group 2 access-control configuration.
- The existing application account has different table permissions depending on the table. For example, event_app has SELECT-only access to the role table, while other tables allow additional operations.
- Therefore, no separate read-only account testing was performed.

6.3 Schema Owner Account

The event_owner account was identified as the owner of the application schema and its database objects.

The privilege inspection confirmed that it has USAGE and CREATE permissions on the school_event_management_db schema. It also owns all 10 application tables and nine sequences.

No separate runtime operation test for event_owner was included in the documented evidence. Its ownership and privileges were verified through PostgreSQL inspection queries.

6.4 Testing Summary

The tests and privilege inspections confirmed the following:

- The event_app account successfully connected to the database.
- The runtime account successfully performed the tested SELECT and INSERT operations.
- UPDATE and DELETE operations were permitted on the event table.
- The runtime account was denied permission to create tables, truncate the event table, and create PostgreSQL roles.
- The INSERT test was rolled back, and verification confirmed that the test record was not retained.
- The event_owner account owns the application schema and its database objects.
- No separate read-only account or Group 2 group-role membership was identified.

6.5 Application Integration Testing

The project uses a Java Spring Boot backend and a React frontend.

The documented tests for this activity focused on direct PostgreSQL connections and database privileges. No separate application API integration test was included in the verified evidence.

7. Issues and Limitations

During the database inspection and access-control testing, several setup considerations and limitations were identified.

7.1 PostgreSQL Environment Setup

PostgreSQL 18 was used to inspect the grp02_event_management_db database.

The database structure was successfully inspected, including its application schema, 10 tables, and nine sequences. The database is owned by postgres, while the application schema and its objects are owned by event_owner.

7.2 Role Configuration and Permissions

The existing PostgreSQL roles were inspected to identify their login capabilities and assigned privileges.

The event_app account uses direct table and sequence privileges instead of membership in a dedicated Group 2 runtime group role. The existing app_runtime_grp and app_admin_grp roles were not modified because they may be shared with other projects.

7.3 Application Integration

The project uses a Java Spring Boot backend and a React frontend.

The documented tests focused on direct PostgreSQL connections and database permissions. A complete application API integration test was not included in the verified evidence.

7.4 Default Privileges

No Group 2-specific default privileges were found in the application schema.

Future database objects may require explicit privilege assignments depending on the account that creates them. The existing permissions and database configuration were documented without modifying other projects.

Despite these limitations, the database structure, existing roles, privileges, and runtime account restrictions were successfully inspected and tested.

8. Evidence and Repository

The SQL script and supporting screenshots for the Group 2 access-control inspection are stored in the group's Car Rental Management System repository under db-access-control/Group02/

Repository links:

- **Repository:** https://github.com/123airabels/Car_Rental_Management_System
- **Group02 access-control folder:** Car_Rental_Management_System/db-access-control/Group02 at main · 123airabels/Car_Rental_Management_System
- **SQL script:** Car_Rental_Management_System/db-access-control/Group02/Group2-roles-and-privileges.sql at main · 123airabels/Car_Rental_Management_System
- **Evidence folder:** Car_Rental_Management_System/db-access-control/Group02/Evidence at main · 123airabels/Car_Rental_Management_System

8.1 Evidence Screenshots

The evidence screenshots cover the following database configurations and tests:

Evidence	Description
Database structure	Application schema and table ownership
PostgreSQL roles	Existing login and group roles
Role memberships	Existing PostgreSQL role memberships
Table privileges	Permissions assigned to application tables
Sequence list	Nine application sequences
Sequence privileges	Runtime account sequence permissions
Schema privileges	USAGE and CREATE permissions
Database privileges	CONNECT, CREATE, and TEMP permissions

Runtime login	Successful event_app connection
SELECT test	Successful retrieval of event records
CREATE TABLE test	Denied permanent table creation
TRUNCATE test	Denied table truncation
INSERT test	Successful INSERT followed by ROLLBACK
Rollback verification	Confirmation that the test record was not retained
Other database access	CONNECT checks for other databases
Database ACL inspection	Existing database access-control settings
Database privilege check	Runtime database permissions
Schema privilege check	Runtime schema permissions
Table privilege summary	Summary of runtime table permissions

8.2 GitHub Upload

The access-control plan, SQL verification script, and evidence screenshots are organized under the Group02 folder.

9. SQL Scripts Summary

The roles-and-privileges.sql script contains the SQL queries used to inspect and verify the existing PostgreSQL access-control configuration.

The script covers the following operations:

- Database Inspection – Checks the current database and connected account.
- Role Inspection – Displays existing PostgreSQL roles and their attributes.
- Role Membership – Checks the membership of login accounts in existing group roles.
- Database Privileges – Verifies CONNECT, CREATE, and TEMP permissions.
- Schema Privileges – Checks USAGE and CREATE permissions on the application schema.
- Table Privileges – Displays SELECT, INSERT, UPDATE, and DELETE permissions.
- Sequence Privileges – Checks the permissions assigned to the nine sequences.

- Object Ownership – Identifies the owners of the application schema and sequences.
- Default Privileges – Checks whether Group 2-specific default privileges exist.
- Runtime Testing – Includes sample operations for checking allowed and denied access.

The script is intended for verification and does not create new roles or modify existing privileges. It documents the actual Group 2 configuration.

10. Verification Results

The database roles and privileges were checked using PostgreSQL inspection queries and direct login tests.

10.1 Runtime Account Verification

Operation	Expected Result	Actual Result
Connect to database	Allowed	Successful
SELECT Records	Allowed	Successful
INSERT	Allowed	Successful; rolled back
UPDATE Event	Allowed	Successful; 0 rows affected
DELETE Event	Allowed	Successful; 0 rows affected
CREATE Table	Denied	Permission denied
TRUNCATE Event	Denied	Permission denied
CREATE Role	Denied	Permission denied

Account: event_app

The runtime account successfully connected to the database and retrieved event records. The INSERT operation was also successful, but the transaction was rolled back to prevent permanent changes.

The CREATE TABLE, TRUNCATE, and CREATE ROLE operations were denied, confirming the account's access restrictions.

10.2 Additional Privilege Verification

The following configurations were also checked:

- The event_app and event_owner accounts have the LOGIN attribute.
- The app_runtime_grp and app_admin_grp roles have the NOLOGIN attribute.
- Neither application login account is a member of the existing group roles.
- The event_app account has CONNECT and TEMP privileges on the database.
- The event_app account has USAGE but not CREATE permission on the application schema.
- The event_app account has direct table privileges based on the requirements of each table.
- The event_app account has USAGE permission on all nine sequences.
- The event_owner account owns the application schema, tables, and sequences.
- The database itself is owned by postgres.
- No Group 2-specific default privileges were found.

These results confirmed the existing access-control configuration of the Group 2 database.

10.3 Additional Security Verification

Additional tests were conducted to verify that the runtime account could not perform restricted operations.

Test Account	Operation	Expected Result	Actual Result
event_app	CREATE TABLE	Denied	Permission denied
event_app	TRUNCATE event	Denied	Permission denied
event_app	CREATE ROLE	Denied	Permission denied

The tests confirmed that the runtime account cannot create permanent tables in the application schema, truncate the event table, or create PostgreSQL roles.

These results demonstrate the separation between normal application operations and restricted database administration tasks.

11. Issues and Fixes

Several application and database issues were identified during testing. The findings were reviewed by the owning group, which provided explanations and proposed fixes for each issue.

Issue Found	Root Cause / Owner's Response	Proposed Fix / Action	Status
Student Number does not enforce the required format.	Only presence, maximum length of 20 characters, and uniqueness were checked.	Apply one format rule in the form, API, and database CHECK constraint.	Accepted
Grade Level accepts unexpected values.	No allowed-values rule was defined in the form, API, or database.	Use a dropdown, API validation, and database CHECK constraint for the school's valid grade levels.	Accepted
Event creation accepts a start time that has already passed today.	Only dates before the current date were rejected.	Reject start times earlier than the current time when the event date is today. Apply the rule during creation and relevant edits.	Accepted

Login accepts uppercase usernames.	Authentication ignores case, while a unique index on lower(username) treats uppercase and lowercase versions as the same account.	Remove case-insensitive authentication and require exact username matching.	Accepted
Overlapping activity schedules display a warning but are still accepted.	Parallel activities within one event are allowed by design. The warning label can make this behavior confusing.	Show a confirmation before saving and change the label to "Runs in parallel with another activity."	Partly Accepted
Deactivated account login attempts show a generic error.	The login failure handler uses one message for all failed attempts to avoid revealing account information.	Show a deactivation message when the username and password are correct but the account is inactive. Keep the generic message for other failures.	Partly Accepted
Event notifications do not appear to refresh after event details are changed.	The reported behavior could not be reproduced. Home and Dashboard lists reload when their pages are opened, but do not refresh while left open.	Refresh the lists when the browser tab regains focus and at regular intervals.	Could Not Reproduce
Database accepts activities scheduled outside their parent event's time range.	The rule was enforced only in application code, with no database-level validation.	Add database triggers to reject invalid activity schedules and event time changes that conflict with existing activities.	Accepted

11.1 Additional Verification Completed

The owning group reviewed the reported issues and provided explanations for the identified behavior.

The following points were clarified:

- Overlapping activities within the same event are allowed because parallel workshops are supported. Venue conflicts between different events are still blocked.
- The generic login error is intentional for security reasons, but a more specific message is proposed for correctly authenticated deactivated accounts.

- The notification refresh issue could not be reproduced as reported. The owning group requested exact reproduction steps and the affected screen.
- Activity schedule restrictions were previously enforced only by application code. A database migration with triggers was proposed to enforce the rule at the database level.

These findings and proposed fixes were documented for monitoring and follow-up with the owning group.

12. Feedback to the Owning Group

The following feedback was provided based on the application testing results and the issues reviewed by the owning group.

Area	Observation	Recommendation
Student Number	The required format was not enforced.	Apply consistent validation in the form, API, and database.
Grade Level	Unexpected text values were accepted.	Restrict entries to the school's valid grade levels.
Event Scheduling	Past start times were accepted for events scheduled today.	Validate both the event date and start time.
Login Security	Uppercase usernames were accepted.	Enforce case-sensitive username authentication.
Activity Scheduling	Overlapping activities may be intentional, but the warning is unclear.	Add confirmation and clarify the warning label.
Account Status	Deactivated accounts receive a generic login error.	Provide a specific message after correct credentials are verified.
Notifications	Lists do not refresh while their pages remain open.	Add automatic refresh when the tab regains focus and at regular intervals.
Database Constraints	Activity schedules can exceed their parent event's time range through direct database operations.	Enforce the restriction using database triggers.

The owning group accepted several findings and identified corresponding fixes. Some findings were partly accepted because the existing behavior was intentional, while the notification issue still requires further reproduction details.

These observations can help improve data validation, user feedback, scheduling accuracy, and database integrity.

13. AI Usage Declaration

ChatGPT was used as a supplementary tool to help understand PostgreSQL roles, privileges, and access control. It also assisted in reviewing SQL queries, explaining errors, preparing verification queries, and organizing the documentation.

The database configuration, query results, runtime tests, and screenshots were based on the actual project environment. The group reviewed the information and used the actual results as the basis for this documentation..

14. Conclusion

The group successfully inspected and tested the existing database access control configuration of the High School Event Management System. The verification covered PostgreSQL roles, database and schema privileges, table permissions, and sequence privileges.

Testing confirmed that the event_app account can perform authorized operations while restricted actions, such as creating tables and truncating records, are denied. The group also identified application and database issues and documented the owning group's responses and proposed fixes.

These results helped the group understand the importance of proper access control, limited privileges, and regular security checks in protecting the database.